

Entity Configurations using Entity Framework Core Fluent API

In this article, I will discuss How to Implement **Entity Configurations using Entity Framework Core (EF Core) Fluent API** with Examples. Please read our previous article discussing [Global Configurations in Entity Framework Core using Fluent API](#) with Examples.

Entity Configurations in Entity Framework Core using Fluent API

In Entity Framework (EF) Core, Entity Configurations allow us to define settings and rules specific to individual entities. This includes mapping domain entities to database tables, specifying primary keys (including composite keys), configuring relationships between entities, and setting up indexes and constraints. By explicitly configuring entities using the Fluent API, we ensure that the database schema aligns precisely with our application's requirements, promoting consistency and maintainability.

Entity configurations are particularly useful in the following scenarios:

- When default conventions are insufficient or inappropriate.
- When we need full control over table mapping, relationships, or constraints.
- When working with existing databases and maintaining consistency between domain models and database schema.

Let us proceed and see some real-time examples to understand how and when to apply entity-level configurations using Entity Framework Core Fluent API in a .NET console application. We will cover the following Entity Configurations:

- [Configuring Table Names and Schema](#)
- [Configuring Primary Keys](#)
- [Configuring Composite Primary Keys](#)
- [Configuring Indexes](#)
- [Configuring Relationships \(One-to-One, One-to-Many, Many-to-Many\).](#)
- [Configuring Cascade Delete Behavior for Specific Relationships](#)
- [Ignoring Entities](#)
- [Configuring Alternate Keys \(Unique Constraints\)](#)
- [Configuring Owned Entities](#)

Configuring Table Names and Schema

By default, EF Core maps each entity to a database table with the same name as the entity's class name or the DbSet property name. However, sometimes, we need to specify a different table name or schema to align with existing database conventions or organizational standards.

This configuration allows for better organization and adherence to naming conventions. For example, to map a Customer entity to a table called tblCustomer in the Admin schema, we configure it as follows. We need to configure the same by overriding the OnModelCreating method of the DbContext class.

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Customer>()
        .ToTable("tblCustomer", schema: "Admin");
}
```

Code Explanations:

- **ToTable("tblCustomer", schema: "Admin"):** Specifies that the Customer entity should be mapped to a table named tblCustomer within the Admin schema.
- **Schema Organization:** Using different schemas (e.g., Admin, Sales) helps organize tables logically within the database.
- **Provider Support:** Ensure that your database provider supports schemas (e.g., SQL Server does, while some others may not).

Configuring Primary Key

Every entity requires a primary key to identify each record uniquely. EF Core follows conventions to automatically detect primary keys, typically properties named Id or <EntityName>Id. If your entity uses a different property as the primary key or if you need to configure composite keys, explicit configuration is necessary. For example, we can set CustomerId as the primary key for the Customer entity as follows:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Customer>()
        .HasKey(c => c.CustomerId);
}
```

Code Explanations:

- **HasKey(c => c.CustomerId):** Specifies that CustomerId is the primary key for the Customer entity.
- **Non-Convention Keys:** Useful when the primary key does not follow EF Core's default naming conventions.
- **Identity Columns:** If needed, you can also configure the primary key to be an identity column (auto-incremented) using ValueGeneratedOnAdd().

Configuring Composite Primary Keys

In some scenarios, an entity may require a composite primary key composed of multiple properties. Composite keys uniquely identify a record using a combination of two or more properties, common in join tables or when no single property is sufficient for uniqueness. For example, an OrderItem entity may use both OrderId and ProductId as a composite primary key:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<OrderItem>()
        .HasKey(oi => new { oi.OrderId, oi.ProductId });
}
```

Code Explanation:

- **HasKey(oi => new { oi.OrderId, oi.ProductId }):** Defines a composite primary key using both OrderId and ProductId properties.
- **Use Cases:** Commonly used in many-to-many relationship join tables or scenarios where the combination of properties ensures uniqueness.
- **Order of Properties:** The order of properties in the composite key matters, especially when creating indexes. Ensure consistency across your configurations.
- **EF Core Limitations:** EF Core does not support composite keys with more than 16 properties. Keep composite keys concise.

Configuring Indexes

Indexes enhance the performance of database queries by allowing faster data retrieval. EF Core allows us to configure indexes on single or multiple properties (composite indexes) at the entity level. You can also enforce uniqueness to ensure data integrity. For example, to create a unique index on the Email property of the Customer entity:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Customer>()
        .HasIndex(c => c.Email)
        .IsUnique();
}
```

Code Explanation:

- **HasIndex(c => c.Email):** Creates an index on the Email property of the Customer entity.
- **IsUnique():** Ensures the Email value is unique across all records, preventing duplicate entries.

Composite Indexes:

You can also create indexes on multiple properties by passing an anonymous object. For example, we want to create an index on the customer entity's first and last name columns. EF Core generates index names by default, but you can also specify custom names using HasDatabaseName.

```
modelBuilder.Entity<Customer>()
    .HasIndex(c => new { c.LastName, c.FirstName })
    .HasDatabaseName("IX_Customer_LastName_FirstName");
```

Filtered Indexes:

EF Core supports filtered indexes (indexes with a WHERE clause) for more advanced scenarios. For example, we can create an Index on the Email where the Where clause [Email] IS NOT NULL as follows. The Email on the WHERE clause will automatically apply the [Email] IS NOT NULL condition to the query.

```
modelBuilder.Entity<Customer>()
    .HasIndex(c => c.Email)
    .HasFilter("[Email] IS NOT NULL");
```

Include Columns:

EF Core allows non-key columns in indexes to be included to cover specific queries. You can consider this a Covering Query. For example, we will create an Index on the Email column, but it will also include the FirstName and LastName columns. This is useful when we want to retrieve the First Name, Last Name, and Email based on the Email condition. In this case, it will directly return the data from the Index table without referring to the actual database table.

```
modelBuilder.Entity<Customer>()
    .HasIndex(c => c.Email)
    .IncludeProperties(c => new { c.FirstName, c.LastName });
```

Configuring Cascade Delete Behavior for Specific Relationships

EF Core allows us to define how deletions in one entity affect related entities. By configuring cascade delete behavior at the relationship level, we can control whether related entities are automatically deleted (Cascade), set to NULL (SetNull), restricted (Restrict), or have no action (NoAction). We can set this delete behavior using the OnDelete()

method. For example, to set up a cascade delete for the Order and OrderItem relationship, we need to configure the settings as follows:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<OrderItem>()
        .HasOne(oi => oi.Order)
        .WithMany(o => o.OrderItems)
        .HasForeignKey(oi => oi.OrderId)
        .OnDelete(DeleteBehavior.Cascade);
}
```

Code Explanations:

- **OnDelete(DeleteBehavior.Cascade):** Configures the relationship so that deleting an Order automatically deletes all related Order Items.

Alternative Delete Behaviors:

- **DeleteBehavior.Restrict:** Prevents deletion of the principal entity if dependent entities exist.
- **DeleteBehavior.SetNull:** Sets the foreign key properties in dependent entities to NULL when the principal is deleted.
- **DeleteBehavior.NoAction:** No action is taken; referential integrity must be handled manually.

Ignoring Entities

There might be scenarios where certain classes in your application represent domain concepts but should not be mapped to database tables. EF Core allows us to exclude these entities from the model, ensuring they do not persist in the database. For example, we have an entity AuditLog that we don't want to map to the database.

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Ignore<AuditLog>();
}
```

Code Explanation:

- **Ignore<AuditLog>():** Excludes the AuditLog entity from the EF Core model, preventing the creation of a corresponding table.

Use Cases:

- **DTOs (Data Transfer Objects):** Classes used for transferring data between layers but not intended for persistence.
- **Base Classes:** Abstract base classes that provide common properties or methods but should not be instantiated.
- **Temporary or Computed Classes:** Classes used for specific operations that do not require storage.

Configuring Alternate Keys (Unique Constraints)

Beyond primary keys, entities can have alternate keys that enforce uniqueness on other properties. Alternate keys act as unique constraints, ensuring that certain properties remain unique across all records. EF Core enforces uniqueness

at the database level, preventing duplicate entries that violate alternate key constraints. For example, to enforce a unique constraint on the Email property of the Customer entity, we can configure as follows:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Customer>()
        .HasAlternateKey(c => c.Email)
        .HasName("AK_Customer_Email");
}
```

Code Explanation:

- **HasAlternateKey(c => c.Email):** Defines Email as an alternate key for the Customer entity.
- **HasName("AK_Customer_Email"):** Assigns a custom name to the alternate key constraint in the database.
- **Use Cases:** Ensuring unique fields like Email, Username, or other business-critical identifiers.
- **Primary vs. Alternate Keys:** An entity can have multiple alternate keys but only one primary key.

Composite Alternate Keys:

We can also define alternate keys using multiple properties as follows. Here, the combination of FirstName, LastName, and DateOfBirth cannot be duplicated.

```
modelBuilder.Entity<Customer>()
    .HasAlternateKey(c => new { c.FirstName, c.LastName, c.DateOfBirth })
    .HasName("AK_Customer_FullNameDOB");
```

Difference Between Alternate Keys and Indexes:

- **Alternate Keys:** Enforce uniqueness, which can be used as foreign keys in other entities.
- **Indexes:** Improve query performance but do not inherently enforce uniqueness unless explicitly specified with IsUnique().

Configuring Owned Entities

Owned entities are types that do not have their own identity and are owned by another entity. The properties of an owned entity are mapped to columns in the owner's table. This configuration is useful for modeling value objects or complex types that logically belong to a single entity. Suppose, Customer has an owned type Address.

```
[Owned]
public class Address
{
    public string Street { get; set; }
    public string City { get; set; }
}
public class Customer
{
    public int CustomerId { get; set; }
    public string Name { get; set; }
    public Address Address { get; set; }
}
protected override void OnModelCreating(ModelBuilder modelBuilder)
```

```
{
    modelBuilder.Entity<Customer>()
        .OwnsOne(c => c.Address);
}
```

Code Explanation:

- **OwnsOne(c => c.Address):** Indicates that Address is an owned entity of Customer.
- **Property Mapping:** The properties of Address will be mapped to columns in the Customer table, reflecting a one-to-one relationship where Address is tightly bound to Customer
- **No Separate Table:** Owned entities do not have their own tables; their properties are included in the owner's table.

Complete Example with Models, DbContext, and Program Class

Let's implement a complete example to understand how these Entity Configurations work together in a .NET console application using Entity Framework Core Fluent API. We will define a few models, apply entity configurations, and show the output.

Creating Models

We will create four entities: **Customer**, **Product**, **Order**, and **OrderItem**. We will also create an enum **OrderStatus** and an owned entity **Address**.

OrderStatus Enum:

Create a class file named **OrderStatus.cs** within the Entities folder and copy and paste the following code.

```
namespace EFCoreCodeFirstDemo.Entities
{
    public enum OrderStatus
    {
        Pending,
        Processing,
        Completed,
        Cancelled
    }
}
```

Address Owned Entity:

Create a class file named **Address.cs** within the Entities folder and copy and paste the following code. This is going to be an owned entity.

```
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    [Owned]
    public class Address
    {

```

```

        public string Street { get; set; }
        public string City { get; set; }
    }
}

```

Customer Entity:

Create a class file named **Customer.cs** within the Entities folder and copy and paste the following code. The following entity represents a customer with personal details and an owned Address.

```

namespace EFCoreCodeFirstDemo.Entities
{
    public class Customer
    {
        public int CustomerId { get; set; } // Primary Key
        public string FirstName { get; set; }
        public string LastName { get; set; }
        public string Email { get; set; } // Alternate Key (Unique)
        public Address Address { get; set; } // Owned Entity
        public ICollection<Order> Orders { get; set; }
    }
}

```

Product Entity:

Create a class file named **Product.cs** within the Entities folder and copy and paste the following code. The following entity represents a product available for purchase.

```

using System.ComponentModel.DataAnnotations.Schema;
namespace EFCoreCodeFirstDemo.Entities
{
    public class Product
    {
        public int ProductId { get; set; } // Primary Key
        public string Name { get; set; }
        [Column(TypeName = "decimal(18,2)")]
        public decimal Price { get; set; }
        public string Description { get; set; }
        public ICollection<OrderItem> OrderItems { get; set; }
    }
}

```

Order Entity:

Create a class file named **Order.cs** within the Entities folder and copy and paste the following code. The following entity represents a customer's order.

```

namespace EFCoreCodeFirstDemo.Entities
{
    public class Order

```

```

{
    public int OrderId { get; set; } // Primary Key
    public DateTime OrderDate { get; set; }
    public OrderStatus Status { get; set; } // Enum stored as string
    public int CustomerId { get; set; } // Foreign Key
    public Customer Customer { get; set; }
    public ICollection<OrderItem> OrderItems { get; set; }
}
}

```

OrderItem Entity:

Create a class file named **OrderItem.cs** within the Entities folder and copy and paste the following code. The following entity represents a line item in an order, linking a Product to an Order.

```

using System.ComponentModel.DataAnnotations.Schema;
namespace EFCoreCodeFirstDemo.Entities
{
    public class OrderItem
    {
        public int OrderId { get; set; } // Composite Key
        public int ProductId { get; set; } // Composite Key
        public Order Order { get; set; }
        public Product Product { get; set; }
        public int Quantity { get; set; }
        [Column(TypeName = "decimal(18,2)")]
        public decimal UnitPrice { get; set; }
    }
}

```

DbContext Class with Entity Configurations using EF Core Fluent API

Next, we need to configure the entity configurations by overriding the OnModelCreating method of the DbContext class. Modify the EFCoreDbContext class as follows.

```

using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            // Configuring the Connection String
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-6P5NK25R\SQLSERVER2022DEV;Database=OrderDB;Trusted_Connection=True;TrustServerCertificate=True");
        }
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            // Configuring the Customer entity
            modelBuilder.Entity<Customer>(entity =>

```

```

{
    // Setting the table name and schema
    entity.ToTable("tblCustomer", schema: "Admin");
    // Setting the primary key
    entity.HasKey(c => c.CustomerId);
    // Setting alternate key (unique constraint) on Email
    entity.HasAlternateKey(c => c.Email);
    // Configuring indexes
    entity.HasIndex(c => c.Email).IsUnique();
    // Configuring the owned entity Address
    entity.OwnsOne(c => c.Address);
    // Configuring One to Many Relationships Between Customer and
    entity.HasMany(c => c.Orders)
        .WithOne(o => o.Customer)
        .HasForeignKey(o => o.CustomerId)
        .OnDelete(DeleteBehavior.Cascade);
});
// Configuring the Product entity
modelBuilder.Entity<Product>(entity =>
{
    // Setting the primary key
    entity.HasKey(p => p.ProductId);
});
// Configuring the Order entity
modelBuilder.Entity<Order>(entity =>
{
    // Setting the primary key
    entity.HasKey(o => o.OrderId);
    // Store the enum as string
    entity.Property(o => o.Status)
        .HasConversion<string>();
    // Configuring One to Many Relationships between Order and OrderItem
    entity.HasMany(o => o.OrderItems)
        .WithOne(oi => oi.Order)
        .HasForeignKey(oi => oi.OrderId)
        .OnDelete(DeleteBehavior.Cascade);
});
// Configuring the OrderItem entity
modelBuilder.Entity<OrderItem>(entity =>
{
    // Configuring composite primary key
    entity.HasKey(oi => new { oi.OrderId, oi.ProductId });
    // Configuring One to Many relationship Between Product and OrderItem
    entity.HasOne(oi => oi.Product)
        .WithMany(p => p.OrderItems)
        .HasForeignKey(oi => oi.ProductId)
        .OnDelete(DeleteBehavior.Restrict);
});
}
// Define DbSets
public DbSet<Customer> Customers { get; set; }

```

```

    public DbSet<Product> Products { get; set; }
    public DbSet<Order> Orders { get; set; }
    public DbSet<OrderItem> OrderItems { get; set; }
}
}

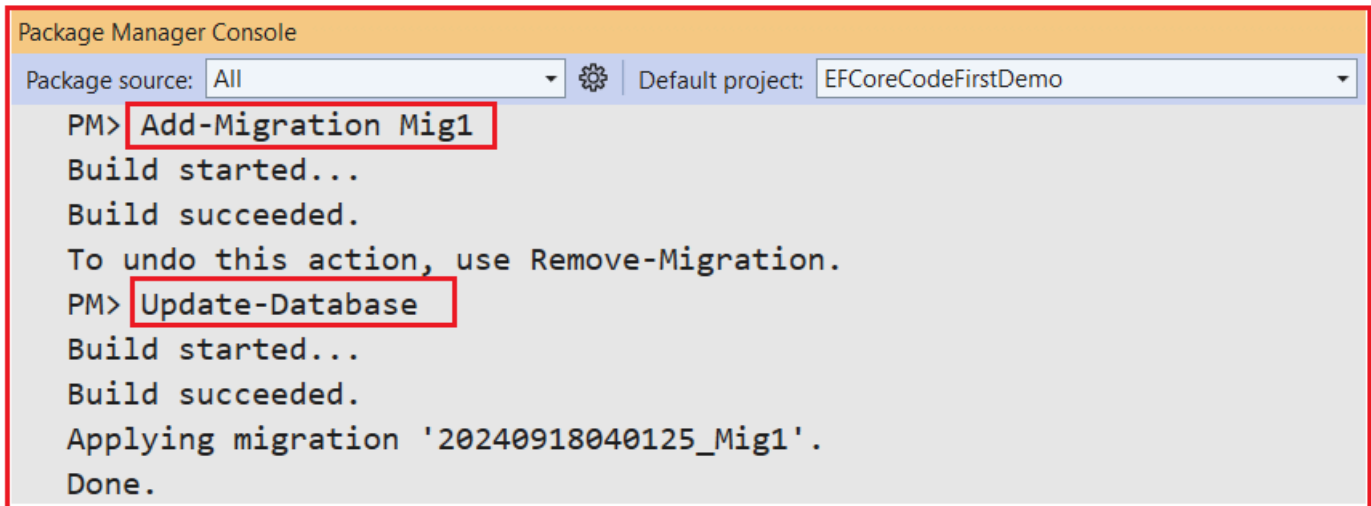
```

Key Points:

- **Table Names and Schemas:** The Customer entity is mapped to a table called tblCustomer in the Admin schema.
- **Primary Keys:** Each entity has a primary key defined.
- **Composite Primary Keys:** The OrderItem entity uses a composite primary key consisting of OrderId and ProductId.
- **Indexes and Alternate Keys:** The Customer entity has a unique index on the Email property.
- **Relationships:** Configured one-to-many relationships between Customer and Order and between Order and OrderItem.
- **Cascade Delete Behavior:** Deleting a Customer will cascade delete their Orders, and deleting an Order will cascade delete its OrderItems.
- **Owned Entities:** The Address type is owned by the Customer entity.

Generating the Migration and Updating the Database

Open the Package Manager Console and Execute the Add-Migration and Update-Database commands as follows.

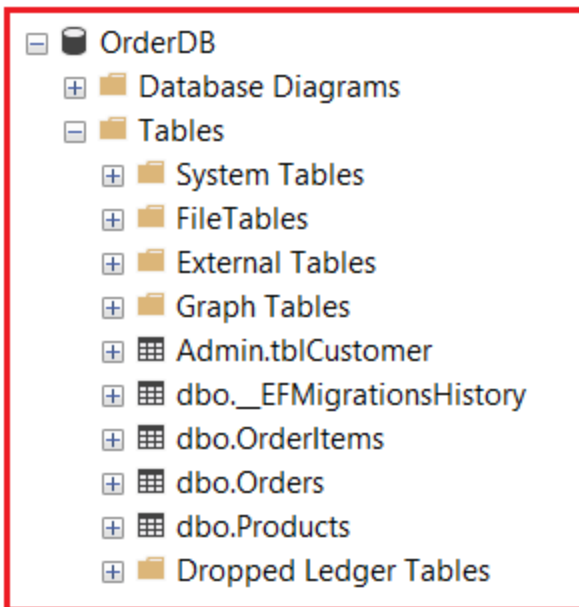


```

Package Manager Console
Package source: All [v] [g] Default project: EFCoreCodeFirstDemo [v]
PM> Add-Migration Mig1
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> Update-Database
Build started...
Build succeeded.
Applying migration '20240918040125_Mig1'.
Done.

```

Once you execute the above commands, the OrderDB database should be created with the Required Customer, Product, Order, and OrderItems table, as shown in the image below.



Testing the Functionalities

Let us create a Program to use the above entities of the Context object and perform the following tasks:

- **Creating Some Products**
- **Creating a Few Customers**
- **Creating Orders with Related Order Items**
- **Displaying All Orders of All Customers**

So, please modify the Program class as follows. The following code is self-explained, so please read the comment lines for a better understanding.

```
using Microsoft.EntityFrameworkCore;
using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {
        static void Main(string[] args)
        {
            // Initialize the database context
            using (var context = new EFCoreDbContext())
            {
                // 1. Creating Some Products
                CreateProducts(context);
                // 2. Creating a Few Customers
                CreateCustomers(context);
                // 3. Creating Orders with Related Order Items
                CreateOrdersWithOrderItems(context);
                // 4. Displaying All Customers All Orders
                DisplayAllOrders(context);
            }
        }
    }
    // Creates and saves a list of products to the database.
```

```

private static void CreateProducts(EFCoreDbContext context)
{
    // Check if products already exist to avoid duplication
    if (context.Products.Any())
        return;
    // List of products to add
    var products = new List<Product>
    {
        new Product { Name = "Laptop", Price = 1200m, Description =
"High-performance laptop" },
        new Product { Name = "Smartphone", Price = 800m, Description
= "Latest model smartphone" },
        new Product { Name = "Headphones", Price = 150m, Description
= "Noise-cancelling headphones" },
        new Product { Name = "Tablet", Price = 500m, Description =
"10-inch display tablet" }
    };
    // Add products to the context
    context.Products.AddRange(products);
    // Save changes to the database
    context.SaveChanges();
    Console.WriteLine("Products have been added to the database.");
}
// Creates and saves a list of customers to the database.
private static void CreateCustomers(EFCoreDbContext context)
{
    // Check if customers already exist to avoid duplication
    if (context.Customers.Any())
        return;
    //Adding List of Customers
    var customers = new List<Customer>()
    {
        new Customer
        {
            FirstName = "Pranaya",
            LastName = "Rout",
            Email = "Pranaya.Rout@DotNetTutorials.NET",
            Address = new Address { Street = "123 Main St", City =
"BBSR" }
        },
        new Customer
        {
            FirstName = "Rakesh",
            LastName = "Kumar",
            Email = "Rakesh.Kumar@DotNetTutorials.NET",
            Address = new Address { Street = "456 Oak St", City =
"Mumbai" }
        }
    };
    // Add customers to the context
    context.Customers.AddRange(customers);
}

```

```

        // Save changes to the database
        context.SaveChanges();
        Console.WriteLine("Customers have been added to the database.");
    }
    // Creates orders with related order items for the customers.
    private static void CreateOrdersWithOrderItems(EFCoreDbContext
context)
    {
        // Check if orders already exist to avoid duplication
        if (context.Orders.Any())
            return;
        // Fetch existing customers
        //Fetch the existing Customer who wants to Place Order
        var customer1 = context.Customers.First(c => c.Email ==
"Pranaya.Rout@DotNetTutorials.NET");
        var customer2 = context.Customers.First(c => c.Email ==
"Rakesh.Kumar@DotNetTutorials.NET");
        // Fetch existing products
        var laptop = context.Products.First(p => p.Name == "Laptop");
        var smartphone = context.Products.First(p => p.Name ==
"Smartphone");
        var headphones = context.Products.First(p => p.Name ==
"Headphones");
        var tablets = context.Products.First(p => p.Name == "Tablet");
        // Create orders for each customer
        var orders = new List<Order>
        {
            new Order
            {
                CustomerId = customer1.CustomerId,
                OrderDate = DateTime.Now,
                Status = OrderStatus.Pending,
                OrderItems = new List<OrderItem>
                {
                    new OrderItem
                    {
                        ProductId = laptop.ProductId,
                        Quantity = 1,
                        UnitPrice = laptop.Price
                    },
                    new OrderItem
                    {
                        ProductId = headphones.ProductId,
                        Quantity = 2,
                        UnitPrice = headphones.Price
                    }
                }
            },
            new Order
            {
                CustomerId = customer2.CustomerId,

```

```

        OrderDate = DateTime.Now.AddDays(-1),
        Status = OrderStatus.Processing,
        OrderItems = new List<OrderItem>
        {
            new OrderItem
            {
                ProductId = smartphone.ProductId,
                Quantity = 1,
                UnitPrice = smartphone.Price
            }
        },
        new Order
        {
            CustomerId = customer1.CustomerId,
            OrderDate = DateTime.Now.AddDays(-2),
            Status = OrderStatus.Completed,
            OrderItems = new List<OrderItem>
            {
                new OrderItem
                {
                    ProductId = tablets.ProductId,
                    Quantity = 3,
                    UnitPrice = tablets.Price
                }
            }
        }
    };
    // Add orders to the context
    context.Orders.AddRange(orders);
    // Save changes to the database
    context.SaveChanges();
    Console.WriteLine("Orders have been created for the customers.");
}
// Displays all orders along with their order items for all
customers.
private static void DisplayAllOrders(EFCoreDbContext context)
{
    // Step 4: Display all orders of all customers
    var customersWithOrders = context.Customers
        .Include(c => c.Orders) //Eager Load Related Order of
Customer
        .ThenInclude(o => o.OrderItems) //Eager Load Related
OrderItems of Orders
        .ThenInclude(oi => oi.Product) //Eager Load Related
Products of OrderItems
        .ToList();
    Console.WriteLine("\n----- All Orders for All
Customers -----");
    foreach (var customer in customersWithOrders)
    {

```



```
Products have been added to the database.
Customers have been added to the database.
Orders have been created for the customers.
```

```
----- All Orders for All Customers -----
```

```
Customer: Pranaya Rout (Pranaya.Rout@DotNetTutorials.NET) Address: 123 Main St, BBSR
```

```
Order ID: 1, Order Status: Pending, Order Date: 2024-09-18 (UTC)
```

```
Order Items:
```

```
    Product Name: Laptop, Quantity: 1, Description: High-performance laptop, Unit Price: $1200.00
```

```
    Total Price for Item: $1200.00
```

```
    Product Name: Headphones, Quantity: 2, Description: Noise-cancelling headphones, Unit Price: $150.00
```

```
    Total Price for Item: $300.00
```

```
Total Order Cost: $1500.00
```

```
Order ID: 3, Order Status: Completed, Order Date: 2024-09-16 (UTC)
```

```
Order Items:
```

```
    Product Name: Tablet, Quantity: 3, Description: 10-inch display tablet, Unit Price: $500.00
```

```
    Total Price for Item: $1500.00
```

```
Total Order Cost: $1500.00
```

```
Customer: Rakesh Kumar (Rakesh.Kumar@DotNetTutorials.NET) Address: 456 Oak St, Mumbai
```

```
Order ID: 2, Order Status: Processing, Order Date: 2024-09-17 (UTC)
```

```
Order Items:
```

```
    Product Name: Smartphone, Quantity: 1, Description: Latest model smartphone, Unit Price: $800.00
```

```
    Total Price for Item: $800.00
```

```
Total Order Cost: $800.00
```

In the next article, I will discuss [Property Configuration in Entity Framework Core using Fluent API](#) with Examples. In this article, I explain **Entity Configurations using Entity Framework Core Fluent API** with Examples. I hope you enjoyed this article on Entity Configurations using EF Core Fluent API.